

Functions in C: Design, and Implementation

Library Functions, Top-Down Design, and User-Defined Functions

Dr. Mohammad Maksood Akhter
Assistant Professor

Department of Computer Science & Engineering

September 22, 2026



Problem

Writing an entire program in one continuous block makes it:

- Hard to read and debug.
- Difficult to reuse common operations.
- Error-prone when updating large codebases.

Solution: Functions

- Divide large programs into smaller tasks.
- Reuse code by calling functions multiple times.
- Improve clarity, testing, and maintainability.

Without Functions

```
1 #include <stdio.h>
2 int main() {
3     int a,b;
4     printf("Enter two numbers: ")
5         ;
6     scanf("%d%d",&a,&b);
7     printf("Sum = %d\n", a+b);
8     printf("Product = %d\n", a*b)
9         ;
10    return 0;
11 }
```

With Functions

```
1 #include <stdio.h>
2 int sum(int, int);
3 int product(int, int);
4 int main() {
5     int a,b;
6     printf("Enter two numbers: ");
7     scanf("%d%d",&a,&b);
8     printf("Sum = %d\n", sum(a,b));
9     printf("Product = %d\n", product(a,b));
10 }
```

Types of Functions

Functions in C are categorized as:

- **Library Functions** – predefined in C libraries.
- **User-Defined Functions** – written by the programmer.

Library Functions	User-Defined Functions
printf(), scanf() sqrt(), pow() abs(), log()	sum(), check_prime() max(), display() factorial(), check_odd_even()

Definition

Library functions are pre-written, tested routines provided by C libraries. They perform common tasks such as I/O, math operations, and string manipulation.

Key Features

- Stored in header files (e.g., `math.h`, `string.h`, `stdio.h`).
- Reused by including headers using `#include`.
- Treated as “black boxes” — used without knowing internal details.

Overview

The `math.h` library in C provides a rich set of mathematical functions for performing computations like powers, roots, trigonometric, logarithmic, and exponential operations.

Including the Library

```
1  #include <stdio.h>  
2  #include <math.h>
```

After including `math.h`, you can directly use its predefined functions.

Commonly Used Functions

Function	Description
<code>sqrt(x)</code>	Returns the square root of <code>x</code> .
<code>pow(x,y)</code>	Returns <code>x</code> raised to the power <code>y</code> .
<code>sin(x)</code> , <code>cos(x)</code> , <code>tan(x)</code>	Return trigonometric values of angle <code>x</code> (in radians).
<code>log(x)</code>	Returns the natural logarithm (base <code>e</code>) of <code>x</code> .
<code>log10(x)</code>	Returns the logarithm base 10 of <code>x</code> .
<code>fabs(x)</code>	Returns the absolute value of a floating number.

Example: Using `math.h` Functions



C Program

```
1 #include <stdio.h>
2 #include <math.h>
3
4 int main() {
5     double a = 25.0, b = 3.0;
6
7     printf("sqrt(25) = %.2f\n", sqrt(a));
8     printf("pow(5, 3) = %.2f\n", pow(5, b));
9     printf("fabs(-7.5) = %.2f\n", fabs(-7.5));
10    printf("log(2.7183) = %.2f\n", log(2.7183));
11
12    printf("sin(0.0) = %.2f\n", sin(0.0));
13
14    return 0;
15 }
```

Program Output

```
sqrt(25) = 5.00
pow(5,3) = 125.00
fabs(-7.5) = 7.50
log(2.7183) = 1.00
sin(0.0) = 0.00
```

Observation

- All functions return double values.
- `math.h` must be linked using `-lm`.
- Angles for trigonometric functions are in **radians**.



Compilation Requirement

- When compiling programs using `math.h` functions, link the math library using `-lm` flag:

```
gcc program.c -o program -lm
```

- The `-lm` option tells the compiler to link against the math library.

Key Points to Remember

- All `math.h` functions operate on and return double values.
- Angles for trigonometric functions must be given in **radians**.
- Convert degrees to radians using:

$$\text{radians} = \frac{\pi \times \text{degrees}}{180}$$

- To use π , define:

```
1 #define PI 3.14159
```

Library Functions

We can use the C library functions `pow()` (power) and `sqrt()` to compute the roots of a quadratic equation in x of the form:

$$ax^2 + bx + c = 0$$

The two roots are defined as:

$$\text{root}_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}, \quad \text{root}_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

C Implementation

```
1      /* Compute the two roots, root_1 and root_2, for disc > 0 */
2      #include <stdio.h>
3      #include <math.h>
4      int main() {
5          double a, b, c, disc, root_1, root_2;
6          printf("Enter coefficients a, b, and c: ");
7          scanf("%lf%lf%lf", &a, &b, &c);
8          disc = pow(b, 2) - 4 * a * c;
9          if (disc > 0.0) {
10              root_1 = (-b + sqrt(disc)) / (2 * a);
11              root_2 = (-b - sqrt(disc)) / (2 * a);
12              printf("Root1 = %.2lf, Root2 = %.2lf\n", root_1, root_2);
13          } else {
14              printf("Complex or equal roots.\n");
15          }
16
17          return 0;
18      }
```

Concept

Top-Down Design is a problem-solving approach where a complex task is divided into smaller, more manageable sub-tasks or modules. Each sub-task is then implemented as an individual function.

Key Idea

- Break a large problem into smaller, simpler steps.
- Each step is implemented as a separate function.
- The main function coordinates these sub-functions.

Concept

User-defined functions are created by programmers to perform specific tasks. They help make code modular, reusable, and easier to debug.

Structure of a Function

A user-defined function in C generally involves three steps:

- 1 Function **Declaration / Prototype**
- 2 Function **Definition**
- 3 Function **Call**

General Form

```
return_type function_name (dummy_argument_list);
```

Examples

```
void sum(void);           // no argument, no return value
int sum(void);            // no argument, returns value
void sum(int, int);       // with arguments, no return value
int sum(int, int);        // with arguments, returns value
```

More Variants

```
int sum(int x, int y);    // value arguments
int sum(int *x, int *y);  // pointer arguments
int *sum(int *x, int *y); // returns pointer
```

General Form

```
1      return_type function_name (formal_argument_list)
2      {
3          /* local variable declarations */
4          /* processing statements */
5          return (value);
6      }
```

Example

```
1      int sum(int x, int y)
2      {
3          int result;
4          result = x + y;
5          return result;
6      }
```

General Form

```
1      variable = function_name(actual_argument_values);
```

Examples

```
1      result = sum(10, 20);  
2      printf("Result = %d", sum(10, 20));
```

Explanation

- The function `sum()` is called from `main()`.
- The actual arguments (10, 20) are passed to the function.
- The returned value is assigned to a variable or printed directly.

Example

```
1      #include <stdio.h>
2      void input();
3      void process();
4      void output();
5      int main() {
6          input();
7          process();
8          output();
9          return 0; }
```

Explanation:

- `input()` – reads data
- `process()` – performs logic
- `output()` – displays result

Advantages

- Improves readability and reusability.
- Easier debugging and maintenance.
- Promotes modular programming.

Problem Statement

Design a program to calculate and display the area of a rectangle using separate functions for:

- Input of dimensions
- Computation of area
- Display of the result

This demonstrates the Top-Down (modular) approach.

Structure Plan

- `main()` — coordinates other functions.
- `getInput()` — reads length and breadth.
- `calculateArea()` — computes area.
- `displayResult()` — prints result.

Example: Modular Design in C (Part 2 – Code)



```
1  #include <stdio.h>
2  float getInput(char varName);
3  float calculateArea(float l, float b);
4  void displayResult(float area);
5  int main() {
6      float length, breadth, area;
7      length = getInput('L');
8      breadth = getInput('B');
9      area = calculateArea(length, breadth);
10     displayResult(area);
11     return 0;
12 }
```

Explanation

- The `main()` function only controls the flow.
- Actual input, computation, and output are done in separate functions.
- This improves modularity and readability.

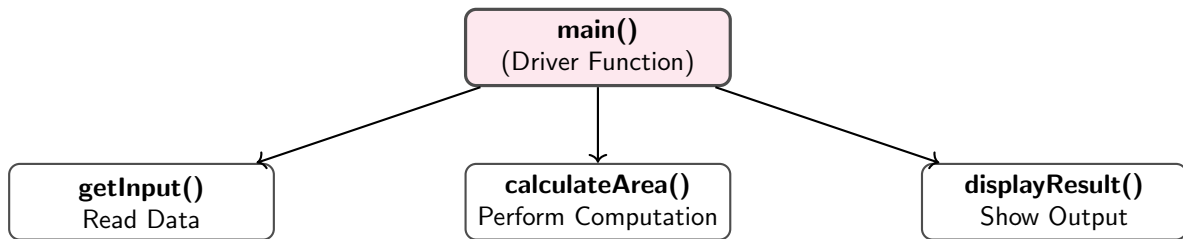
Example: Modular Design in C (Part 3 – Functions Implementation)



```
1 float getInput(char varName) {  
2     float value;  
3     printf("Enter %c: ", varName);  
4     scanf("%f", &value);  
5     return value;  
6 }  
7 float calculateArea(float l, float b) {  
8     return l * b;  
9 }  
0 void displayResult(float area) {  
1     printf("Area = %.2f\n", area);  
2 }
```

Key Takeaways

- Each function performs a single, clear task.
- The program is easier to test, debug, and reuse.
- Top-Down design promotes modular, maintainable code.



Each function executes independently, returning control to **main()**.



Understanding the Flow

- The **main()** function controls overall program logic.
- Each sub-function (`getInput`, `calculateArea`, `displayResult`) performs a specific task.
- After execution, control always returns to **main()**.

- Functions divide large problems into smaller reusable modules.
- Two main types: **Library** and **User-Defined**.
- Top-Down Design promotes modular, maintainable programs.
- Header files like `math.h` contain predefined reusable code.

Thank You

Questions?